

# GDN Tree-Scan: Design and Optimization of Tree Verification for Recurrent-Hybrid Language Models

Zhiyuan Ma

zhiyuanma0520@gmail.com

**Abstract**—Tree speculative decoding for long-running coding agents must preserve coherent continuation across repeated generation and tool calls. We present GDN Tree-Scan, a tree-verification system for Gated DeltaNet hybrid models. It separates branch-local computation from persistent state: independent paths are verified in parallel, and accepted-path replay reconstructs recurrent state while convolution history and attention entries are gathered from the same path for continuation. A shared logical tree coordinates these operations. Fused candidate selection, GPU-resident acceptance, and grouped split-K attention reduce repeated work and host orchestration. On Qwen3.8-27B NVFP4 on GB10, among runs producing nonempty patches on both shared SWE-bench Verified Astropy tasks, the best recorded tree run achieves 29.09 pooled tokens/s versus SGLang EAGLE’s 26.89, an 8.18% higher rate. In a separate ten-task deployment, all tree-served agent attempts terminated and produced nonempty patches, with no degeneration detected by the recorded trace checks. Component tests verify candidate-selection parity and measure attention repeatability and numerical error.

**Index Terms**—speculative decoding, Gated DeltaNet, state commitment, numerical reproducibility, language-model serving

## I. INTRODUCTION

Speculative decoding reduces serial target-model invocations by verifying draft tokens in parallel. Its sampling guarantee assumes that verification supplies the intended target probabilities. Tree speculation expands the candidate set, but it does not relax that assumption [1]–[3]. For recurrent-hybrid models, realizing the verifier requires more than an attention mask: a candidate must inherit the state of its ancestors, and the next serving iteration must inherit exactly the path selected for continuation [4], [5].

GDN Tree-Scan turns a candidate token tree into a target-model forward and a native continuation state. Its design connects an ancestry-aware attention path, branch-local GDN computation, a device acceptance walk, and accepted-path publication across recurrent, convolution, and attention key-value (KV) storage. A shared descriptor carries the logical tree through each stage [6], [7]. This system boundary matters because a verifier can compute plausible candidate logits yet publish the wrong state for the next iteration.

The central execution choice is to verify branches in temporary state and replay only the selected path into the native running state. The verifier scans branch paths, while the fixed-shape committer replays accepted operands through native GDN updates in a captured graph. Their finite-precision agreement is a separate requirement, not assumed from the recurrence. This choice exposes concrete optimization questions: how much branch state to cache, when to recompute ancestry, how

to organize independent GPU work, and how to keep full-vocabulary probability arithmetic on the device. The attention layout and accepted-state mappings constrain those changes as much as the recurrent equation does.

The paper studies three coupled design questions in recurrent-hybrid tree verification. *First*, how can branch-local computation produce one coherent continuation across recurrent, convolution, and attention state? We define a shared logical tree and coordinate state publication through its logical-to-physical mapping. *Second*, how should verification trade temporary state storage against repeated computation? We use path-parallel scans with shared boundary states and replay the accepted path into persistent state, building on established chain-scheduling and replay mechanisms [8], [9]. *Third*, how can this organization reduce serving overhead? We combine fused candidate selection, device-resident acceptance, and grouped split-K attention with a spine-contiguous KV layout. Component validation and coding-agent workloads characterize the resulting system; matched ablations remain necessary to attribute gains to individual choices.

The target workload is a long-running coding agent: generation alternates with repository inspection, tool execution, and tests, and a task can require many model requests. Performance therefore depends on useful completed work as well as token production. Our workload evaluation uses explicitly identified Astropy tasks from SWE-bench Verified [10]. We report the model, serving route, task cohort, and measurement definition together; a decode-rate change is not a task-completion speedup. Candidate-selection and attention tests complement the agent-workload measurements.

Tree verification and compact accepted-state reconstruction have direct prior art. Section II positions our implementation against those mechanisms; we do not claim the first hybrid-tree system, a new GDN recurrence, or the invention of accepted-path replay or chain decomposition. The architecture and kernel design are developed in Sections III–IV, and Section V separates optimization mechanisms from their measured scope. Measurement provenance is documented in Appendix A.

## II. BACKGROUND AND RELATED WORK

### A. Drafting, verification, and serving

Speculative sampling constructs an exact target sample from proposal and residual distributions, conditional on the probabilities actually supplied [1], [2]. SpecInfer supplies the tree-verification framing; Sequoia, EAGLE-2, and OPT-Tree investigate tree construction and its performance conse-



Fig. 1. Tree verification and accepted-path commitment. (a) Drafting provides a candidate tree and proposal probabilities  $q$ . (b) Branch-local computation produces target probabilities  $p$  and retains temporary state. (c) Sampling selects a materialized path (here  $a, d$ ); recurrent replay, convolution gathering, and KV remapping publish  $\mathcal{S}(mad)$ . The newly sampled token  $b^*$  remains pending. One descriptor keeps logical ancestry and physical storage aligned.

quences [3], [11]–[13]. EAGLE-3 and DFlash further change the drafter, through feature-conditioned autoregression and block-parallel diffusion proposals respectively [14], [15]. These drafting mechanisms are distinct from the recurrent verifier and state-publication choices studied here.

PagedAttention and SGLang address the scheduling, memory, and execution substrate [16], [17]. Marconi addresses prefix-cache admission and eviction for hybrid models [18]. Hybrid serving additionally manages recurrent-state pools alongside KV storage, as documented by SGLang’s hybrid-model integration [19], [20]. Thus neither a fast isolated recurrent kernel nor low transient state memory alone determines user-visible latency. Flash-Decoding splits the KV sequence and combines partial attention results; FlashInfer develops customizable attention with head/query fusion and split-KV execution; DeFT specializes KV grouping and partitioning to trees [21]–[23]. Our grouped split-K path adapts established work-partitioning techniques. The system uses a modified FlashAttention-2 kernel for tree attention, with native-compatible prefill and a specialized fixed-row decode kernel. Its physical layout and work partitioning are part of the verifier design [6], [24].

### B. Recurrent and hybrid speculation

Gated DeltaNet augments a decayed recurrent state with a data-dependent delta update [4]. Parallel delta-rule formulations provide relevant algebraic background [25]. Snakes and Ladders introduces activation replay for SSM speculative decoding, and STree also uses replay to recover continuation state [5], [26]. Stateful speculation is established territory: STree studies speculative trees for hybrid state-space models, SpecMamba targets an FPGA implementation, and component-aware self-speculation studies how hybrid components affect the design [5], [27], [28]. These works motivate an explicit state lineage; they do not justify treating every recurrent update or numerical implementation as interchangeable.

### C. Direct GDN comparisons

**Trees from Marginals (Weaver)** combines a factorized drafter with an autoregressive adapter. Its GDN verifier uses an ancestor-masked triangular solve and replays the selected path through a short recurrence at commit time; padded verification shapes support CUDA graphs [9]. Thus read-only speculative verification followed by accepted-path replay is direct prior art, even when the verification arithmetic differs from ours.

**SpecLA** includes state-resident, value-tiled serial verification and a chain-decomposed tree schedule. Chains export boundary states for dependent chains, while independent ready chains run in parallel. This directly overlaps our path groups and temporary cut states. Its state policy retains accepted factors for a later fused update, whereas our state-publication stage replays native GDN updates into running rows [8]. The difference is an implementation choice whose costs require measurement.

**Bole** derives a tree-structured closed form and evaluates it with a value-tiled finite-polynomial solver. It stores compact update factors and reconstructs the accepted state. Its SGLang integration combines batch-wide verification budgeting with captured GPU execution. Evaluation includes unquantized Qwen3.5-27B on GB10 and online replay of agent sessions [29]. Its state and serving mechanisms directly overlap the problem studied here. The cost of our replay implementation is not a lower bound on alternatives.

Concretely, Bole’s correction system is  $(I + G)U = R$ , where  $G$  contains strict-ancestor interactions and  $R$  is the source correction. Tree depth  $d$  gives  $G^{d+1} = 0$ , so  $U = \sum_{m=0}^d (-G)^m R$ . This finite identity changes the arithmetic schedule from sequential updates to matrix propagation [29]. Our path preserves sequential updates and replays accepted operands; Section V-D explains why the numerical comparison remains an empirical question.

**TreeWY** expresses GDN tree verification as a triangular system and reconstructs the accepted state from pseudo-values. Its vLLM evaluation uses Qwen3.5-35B and 397B on B200, greedy verification, and disabled prefix caching. It explicitly

TABLE I  
DIRECT MECHANISM-COMPARISON SCOPE. THE AUTHORS’  
IMPLEMENTATIONS OF THE EXTERNAL METHODS LISTED HERE WERE NOT  
EXECUTED IN THIS STUDY.

| Method                                    | Overlapping mechanism                                                          | Boundary for this study                                                                                 |
|-------------------------------------------|--------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Trees from<br>Marginals [9]<br>SpecLA [8] | Tree solve; selected-path<br>replay<br>Chain decomposition;<br>boundary states | Different verifier<br>arithmetic; no matched run<br>Direct schedule overlap;<br>different commit policy |
| Bole [29]                                 | Parallel verifier; factorized<br>commit                                        | Different stack and<br>precision; no matched run                                                        |
| TreeWY [30]                               | Tree solve; accepted-state<br>reconstruction                                   | Different models and<br>operating regime; no<br>matched run                                             |
| ReplaySSM [31]                            | Input caching; deferred<br>state work                                          | Alternative state policy;<br>no matched run                                                             |
| GDN Tree-Scan                             | Scan/replay; accepted-path<br>remapping                                        | Component validation and<br>coding-agent workloads                                                      |

permits finite-precision disagreement with the baseline and reports that wider trees improve acceptance without yet winning throughput [30]. Its numerical contract and hardware differ from our evaluated system.

**ReplaySSM** caches recurrent inputs and changes when state is materialized; its implementation RFC includes GDN and Mamba2 [31]. Reconstruction and replay are not claimed as new concepts here. Table I states the comparison boundary. Published speed ratios from different checkpoints, precision formats, workloads, and software builds are not combined into a performance ranking.

#### D. Scope of the system contribution

The accepted-prefix condition is a correctness requirement, not a new sampling law. Chain scheduling, replay, GPU-resident acceptance, graph capture, and split-K attention each have precedents. Our contribution is their concrete coordination with a logical-to-physical KV map and native continuation state in the reported deployment. In particular, spine-first KV placement is a specific layout intervention; general floating-point reduction sensitivity and batch-invariant attention are already established [32]. The current evidence does not establish priority for this intervention or a general numerical advantage over compact-state verifiers.

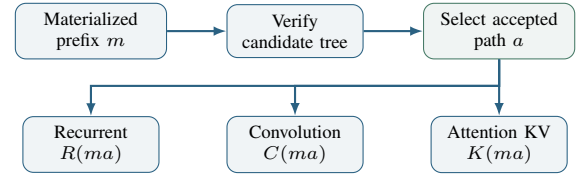
### III. TREE-VERIFIER ARCHITECTURE

#### A. State and the execution boundary

Let  $m$  be a materialized token prefix: the tokens already consumed by the target’s forward computation. Write its persistent state as

$$S(m) = (R(m), C(m), K(m)), \quad (1)$$

where  $R$  collects recurrent matrices,  $C$  collects convolution histories, and  $K$  collects attention KV entries. The logical emitted prefix can also include a newly sampled token  $b$  that has not yet been consumed. The continuation boundary is then  $(S(m), b)$ , not a claim that  $S(mb)$  has already been computed. This distinction is essential when counting the correction or bonus token while checking cache state.



Same selected materialized prefix on all three surfaces;  
correction token  $b$  may remain pending for the next forward.

Fig. 2. The accepted-prefix interface. Recurrent, convolution, and attention-KV state must all describe the same materialized prefix. A newly sampled correction or bonus token can remain pending until the next forward.

A tree descriptor contains candidate token IDs, parent indices, depth/position information, attention visibility, and maps between logical nodes and physical slots. For a candidate node  $i$ , each target layer must consume the state obtained from the candidate’s root-to-parent path. At publication, the selected materialized path, pending-token convention, and next-drafter input row must agree. Figure 2 illustrates the three state surfaces coordinated by the verifier [6], [7].

#### B. The recurrence and path locality

For one GDN head, let  $R \in \mathbb{R}^{d_v \times d_k}$ . Queries and keys are in  $\mathbb{R}^{d_k}$  and values in  $\mathbb{R}^{d_v}$ . With decay  $\alpha_i$  and write strength  $\beta_i$ , a node applies the standard update [4]

$$R'_i = \alpha_i R_{\pi(i)}, \quad \delta_i = \beta_i (v_i - R'_i k_i), \quad (2)$$

$$R_i = R'_i + \delta_i k_i^\top, \quad o_i = R_i q_i, \quad (3)$$

where  $\pi(i)$  is its parent. A previous node in the packed buffer need not be  $\pi(i)$ . The causal convolution likewise requires path predecessors rather than adjacent packed rows. The attention visibility relation must select the same logical prefix and ancestry. These requirements follow from the model’s computation; a correct attention mask cannot repair a wrong recurrent parent.

The implementation verifies nodes with branch-local scratch and makes the selected continuation durable through replay and remapping. This is an implementation route, not the only way to satisfy the contract [6], [7]. In particular, rejecting a branch requires that its contents cannot influence future computation; it does not require zeroing every scratch byte if future reads are correctly restricted.

#### C. A descriptor shared by verification and commitment

Figure 1 shows how a shared descriptor connects drafting to continuation. Native multi-token prediction (MTP) supplies a principal chain and runner-up candidates; suffix prediction extends selected paths within a bounded candidate capacity [33], [34]. Combining retrieval and neural drafting has prior art, including SAM Decoding [35]. The verifier derives attention visibility, physical packing, path scheduling, and the acceptance walk from the same logical topology. Padding permits reusable execution shapes while remaining invisible to attention and selection. The particular tree geometry used in our experiments is specified in Section VI.

---

**Algorithm 1** Logical verifier and continuation boundary

---

**Require:** Reference-aligned state boundary, candidate descriptor  $T$

- 1: Map logical candidates to physical slots and positions.
  - 2: Verify using path-local convolution, GDN, and attention.
  - 3: Apply the configured sampler to matching target/proposal rows.
  - 4: Obtain the accepted path and any pending correction token.
  - 5: Publish recurrent and convolution state for the materialized path.
  - 6: Map its attention KV entries to the native continuation layout.
  - 7: Map the selected hidden-state row to the next drafter input.
  - 8: Preserve the native pending-token and position convention.
  - 9: **return** Continuation boundary and emitted-token accounting
- 

The attention layout places the principal spine first in the physical suffix and permutes KV write locations and ancestry-mask key columns while retaining logical query-row and position order. Selection remains in logical-node space; accepted-path publication maps those nodes back to their physical slots. This distinction prevents a layout optimization from silently changing which key/value entries a branch reads. Recurrent paths begin at the native pre-tree state or a transient cut state; the device committer publishes the selected recurrent and convolution state into native running rows and re-linearizes its attention KV entries [6], [7].

#### IV. GPU SCAN AND ACCEPTED-PATH REPLAY

##### A. Path-parallel verification and state lifetime

The GDN verifier decomposes the tree into dependent path groups, a scheduling family also studied by SpecLA [8]. A first path starts from the request’s pre-tree state and exports the cut states needed by downstream paths. The second level evaluates independent branch paths from those cut states. Each path applies sequential node updates in ancestry order while separate programs cover value tiles and heads. Candidate outputs continue through the model; rounded operands are recorded for later accepted-path replay [6].

Cut states and operand rings are temporary within a verification step. The durable continuation resides in the request-owned native recurrent and convolution running rows. Replay starts from the pre-tree running state and publishes the accepted frontier back to those rows; prefix-cache snapshots and restores use that native layout. Static work buffers can be reused by CUDA graphs, but they do not replace the request’s persistent state with a branch-leaf pointer.

The two levels execute in separate GPU launches and exchange only the boundary states required by downstream paths. This schedule exposes parallel branch work while retaining the state-transfer cost at the decomposition boundary.

---

**Algorithm 2** Logical recurrence and selected-path publication. The implementation groups independent paths into GPU launches and retains only their needed cut states.

---

**Require:** Pre-tree state  $R_0$ , descriptor  $T$ , rounded node operands  $x_i$

- 1: **for** node  $i$  in topological order **do**
  - 2:  $R_p \leftarrow R_0$  if  $i$  has no candidate parent; otherwise  $R_p \leftarrow H[\pi(i)]$
  - 3:  $(H[i], o_i) \leftarrow \text{VerifyStep}(R_p, x_i)$
  - 4: **end for**
  - 5: Finish target forward and select accepted materialized path  $A$
  - 6:  $R \leftarrow R_0$
  - 7: **for** node  $i$  in  $A$ , in path order **do**
  - 8:  $(R, \_) \leftarrow \text{NativeUpdate}(R, x_i)$
  - 9: **end for**
  - 10: Publish  $R$  and matching convolution history to native running rows
  - 11: Remap accepted KV entries and retire speculative scratch
  - 12: **return** Matching continuation state and any pending token
- 

##### B. Sequential arithmetic and accepted-path replay

Verification uses sequential path scans with fp32 carried state, raw gate inputs, and in-kernel query/key normalization. Accepted-path replay uses native fused GDN updates with recorded accepted operands and in-place publication to the fp32 state bank. Its graph contains one native update call per GDN layer. Query/key normalization, beta conversion, gate evaluation, and output casts can differ between these implementations; agreement is not guaranteed by their common real-arithmetic recurrence [6].

The sampling stage produces accepted node indices and path lengths on the GPU. Recurrent replay, convolution publication, and attention-KV publication consume this same path. Graph capture reduces host launch overhead for the sequence of per-layer native updates; each layer retains its own update kernel.

##### C. Temporary state versus recomputation

Path decomposition caches the states needed where downstream paths branch, then reuses them across independent work. If a program exports  $N_c$  cut states with value-tile width  $B_v$  and key dimension  $d_k$ , their fp32 payload is

$$M_{\text{cut}} = 4N_c B_v d_k \text{ bytes.} \quad (4)$$

This is a logical payload, not a register-allocation or total-memory measurement. Recomputing ancestry could remove some exports at the cost of repeating updates. Exporting every candidate state would instead trade more storage for less recomputation. The two-level path schedule occupies a particular point in this design space.

Accepted-path replay carries one state tile through a bounded path loop. Its carried-state footprint is  $O(B_v d_k)$  in tree size, while the recorded operands and candidate outputs scale with the verification shape. Static padding makes graph buffers and kernel geometry reusable, but padded rows must remain

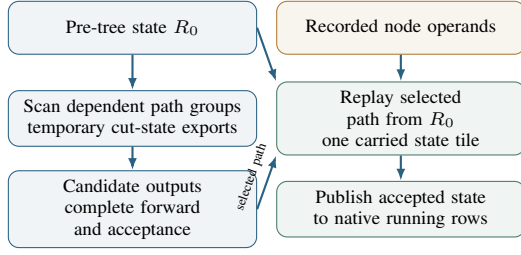


Fig. 3. State lifetime in the path-scan verifier. Temporary cut states connect path groups; recorded operands support selected-path replay. Only the accepted frontier becomes native persistent state.

invisible to attention and selection. Neither a shorter logical path nor fewer state writes alone establishes a faster complete request.

Compact triangular-solve and finite-Neumann methods choose another point in this design space: they form correction factors and reconstruct state without replaying every accepted rank-one update. Their arithmetic schedule differs from sequential scan and replay. Lower state storage or less replay work alone does not establish lower complete-step latency, and a real-arithmetic identity does not settle finite-precision agreement [29], [30].

## V. OPTIMIZATION MECHANISMS AND BACKEND CONSTRAINTS

### A. Reuse logits and fuse full-vocabulary selection

The drafter uses the full vocabulary. At each head depth, a single logits tensor supplies the principal token and runner-up candidates. The principal chain advances through captured MTP iterations; sibling proposals reuse the head scores without independent drafter forwards. The suffix predictor extends selected paths within the same fixed verification capacity [6].

A fused CUDA selection kernel emits both the spine token and the three ordered candidate indices directly into static graph buffers. It replaces separate argmax/top-k reductions and their output copies; it does not remove the vocabulary projection itself. Matching the reference implementation requires preserving two tie conventions: argmax chooses the lowest token index among equal maxima, while top-k has its own ordering among selected ties. The fused kernel reproduces both conventions, including duplicate proposals when the reference produces them. Changing that policy would change the proposal tree, so it is a separate algorithmic decision.

### B. Keep acceptance and publication on device

The tree-acceptance walk consumes full-vocabulary target/proposal probabilities and produces accepted node indices, emitted tokens, path lengths, and the pending correction or bonus token. Device-side product filling feeds the captured committer, avoiding host construction of per-layer path lists between selection and publication [6].

Recurrent replay, convolution publication, attention-KV remapping, and the next drafter’s hidden-row selection all consume that same path. CUDA graphs reduce repeated launch

orchestration; they do not alter the required selected-prefix contract.

### C. Prepare convolution and replay operands for the fixed shape

Tree convolution gathers a causal window for each candidate from its ancestors and the prior native convolution state. The verifier uses static path-index preparation, fused tree convolution, and stacked operand rings spanning the GDN layers. Ordered tap operations and cast boundaries remain part of its arithmetic contract. State publication selects the accepted convolution window and writes it to the native running row; graph scratch is prepared for the next iteration rather than becoming durable per-node history [6].

### D. Spine-first layout and grouped split-K attention

The modified FlashAttention-2 kernel applies tree visibility inside attention. Its spine-first packing keeps the principal path contiguous in the candidate suffix. As in prior work on batch-invariant inference, physical reduction order matters [32]. Here the relevant intervention is tree-specific: inserting masked branch columns between spine keys can change the reduction layout of an online softmax. Permuting KV slots and the matching ancestry-mask key columns preserves logical query order while controlling the physical key layout [6].

The attention kernel shares staged key/value tiles across grouped query heads. Split-K partitions the context, and each partition computes a local maximum, denominator, and weighted-value sum. A combine kernel rescales and merges these partial results. This exposes more independent work at batch one, but changes floating-point accumulation order. We therefore evaluate repeatability and numerical error against both the reference attention kernel and a high-precision reference.

Prefill uses a native-compatible FlashAttention-2 path; tree verification uses the ancestry-aware decode kernel. The physical slot map is shared by attention writes, masked reads, and accepted-path publication.

### E. Publish one coherent native continuation

If accepted logical node  $a_j$  becomes continuation position  $j$ , the KV remapper copies from the physical slot assigned to  $a_j$  into the request’s native destination slot for  $j$ . The slot permutation is applied to verification reads and writes as well as publication. A valid ancestry mask alone cannot repair inconsistent addressing.

Recurrent and convolution publication make the same selected frontier visible in native running rows. Prefix caching and full/piecewise CUDA graphs are enabled in the reported deployment. Their state ownership follows the native request lifecycle; temporary cut-state exports remain a distinct within-step resource.

## VI. EVALUATION SETUP

Performance is evaluated on coding-agent workloads, with the task subset and serving configuration attached to every reported rate. The Qwen3.8-27B NVFP4 evidence uses SWE-bench Verified Astropy tasks on a single GB10/DGX Spark: a

TABLE II

OPTIMIZATION MECHANISMS AND THEIR VALIDATION. WORK ADDRESSED DESCRIBES THE DESIGN OBJECTIVE; WORKLOAD MEASUREMENTS EVALUATE THEIR COMBINED EXECUTION.

| Mechanism                           | Work addressed                                        | Design                                                       | Validation scope                                |
|-------------------------------------|-------------------------------------------------------|--------------------------------------------------------------|-------------------------------------------------|
| Single-logits drafting              | Repeated projection and score preparation             | Full-vocabulary scores reused across branches                | Enabled in evaluated configuration              |
| Fused top-3 selection               | Separate argmax/top-k reductions and copies           | One CUDA selection into graph buffers                        | Exact selection and graph-replay tests          |
| Path-parallel GDN scan              | Serial branch work and repeated ancestry              | Two-level path schedule with transient cut states            | Execution configuration documented              |
| Device acceptance and commit        | Host path construction and launch orchestration       | GPU walk, path indices, captured replay                      | Execution confirmed in workload runs            |
| Convolution and operand preparation | Repeated path-index and per-layer preparation         | Fused convolution, pregather, stacked layer rings            | Enabled in evaluated configuration              |
| Spine layout and KV remap           | Inconsistent physical ancestry and continuation       | Spine-first permutation plus accepted-path remap             | Common slot mapping in verification/publication |
| FA2 grouping and split-K            | Repeated KV staging and insufficient independent work | Shared KV staging; context partitions and combine            | Repeatability and numerical error checks        |
| Native-state lifecycle              | Persistent branch-specific cache ownership            | Publish native running rows; graphs and prefix cache enabled | State consistency and recorded agent behavior   |

shared two-task comparison of three tree runs against SGLang EAGLE, plus a separate ten-task tree deployment [6], [10]. The best recorded shared-task rate supplies the headline; later runs and the larger deployment remain in the evaluation. These selected subsets are not the full benchmark. Every decode-rate result uses the pooled completed-request estimator in Eq. 5, excluding first-output latency. Recorded agent duration includes the agent loop, model service, tools, and repository operations; server startup and subsequent evaluator time are separate.

The evaluated tree has 27 draft candidates and one root, padded to 32 verification rows. A five-depth MTP head supplies 15 candidates; suffix prediction adds a six-token principal tail and rescue branches of four and two tokens. Maximum draft depth is eleven, and four post-root MTP forwards construct the head. Attention groups query heads in pairs and divides the context into four partitions. The model has 48 GDN layers and sixteen full-attention layers. All reported tree runs use this geometry and the patched FlashAttention-2 path; exact source and kernel versions are retained in the artifact.

## VII. RESULTS

### A. SWE-bench Verified coding-agent workload

Our workload consists of Astropy instances from the SWE-bench Verified test split. The agent uses Qwen Code 0.19.4 to inspect repositories, edit files, execute tools, and produce patches. A shared two-task comparison covers three tree runs and one SGLang EAGLE run that each produced nonempty patches on both tasks. A separate ten-task segment, Cqc10, recorded on 24 August 2026, supplies the latest complete single-configuration deployment case. The task/evaluation worker is x86-64; model serving runs on the GB10. These are full agent attempts. Table III lists every task and saved evaluator outcome in Cqc10.

*a) Ten-task deployment:* The served model is the RadixArk Qwen3.8-27B NVFP4 checkpoint with ModelOpt mixed quantization and bfloat16 execution dtype. The tree verifier uses the geometry and attention configuration specified

TABLE III

ALL TEN TASKS IN THE COMPLETED CQC10 ASTROPY CASE. IDS HAVE PREFIX `ASTROPY__ASTROPY-`. TIMES ARE AGENT ELAPSED MINUTES, ROUNDED FROM SAVED RUNNER METADATA; EVALUATION IS SUBSEQUENT. THIS SELECTED DEVELOPMENT SEGMENT HAS ONE ATTEMPT PER LISTED TASK AND NO MATCHED NATIVE ARM.

| Task suffix | Agent time (min) | Saved evaluation |
|-------------|------------------|------------------|
| 13977       | 24.59            | Failed tests     |
| 14096       | 22.99            | Resolved         |
| 14182       | 7.23             | Failed tests     |
| 14309       | 1.67             | Resolved         |
| 14365       | 6.78             | Failed tests     |
| 14369       | 27.70            | Resolved         |
| 14508       | 41.24            | Resolved         |
| 14539       | 18.57            | Resolved         |
| 14598       | 25.79            | Failed tests     |
| 14995       | 4.59             | Resolved         |

in Section VI, with full-vocabulary drafting and reused draft logits. Prefix caching and full/piecewise CUDA graphs are enabled. The engine permits one active sequence, with maximum context 131,072 tokens and a 4,096-token scheduler budget. Source commit `e7af6b595a8b` and the loaded extension hash are retained in the artifact. The implementation uses the graph/cache-enabled stochastic route described above.

The proxy supplies temperature 0.6, top- $p$  0.95, top- $k$  20, minimum- $p$  0, and presence penalty 1.0; the engine seed is zero. The per-response output cap is 24,000 tokens, with a separate 20,000-token compaction cap and a 9,000-second agent timeout. Proxy auto-continuation is disabled. The agent runs in the task's repository image on an internal network that permits the model proxy and blocks external egress; per-task records retain the verified network-policy fingerprint. Agent duration comes from the task runner's elapsed-time field: it includes the agent loop and tool work, and excludes server startup and subsequent evaluation. We do not infer a completed-task speedup from model-only counters.

b) *Workload observations*: All ten attempts have evaluator reports: six resolved their tasks and four failed tests. Recorded agent durations sum to 181.15 minutes, with a range of 1.67–41.24 minutes. This is an observed task ledger, not a representative SWE-bench Verified success estimate. The sum is accumulated agent time, not elapsed campaign time including startup and evaluation.

Every task used tools and produced a nonempty patch. Analysis of the recorded traces detects no degeneration in these ten runs and no malformed tool-call arguments. The degeneration heuristic jointly requires at most 200 visible characters, zero tool calls, and at least 2,000 thinking characters; none of the ten traces meets that conjunction. This screen can miss loops that continue narrating or using tools, so its zero detections establish neither universal degeneration freedom nor preserved task quality. The earlier campaign’s detected degeneration remains an adverse observation in the lineage audit.

c) *Pooled decode comparison*: For completed request  $r$ , let  $n_r$  be its output-token count,  $\ell_r$  its server-recorded end-to-end latency, and  $f_r$  its time to first output. With  $R$  completed requests and  $N = \sum_r n_r$ , we use

$$D_{\text{pool}} = \frac{\sum_{r=1}^R (n_r - 1)}{\sum_{r=1}^R (\ell_r - f_r)} = \frac{N - R}{\sum_r \ell_r - \sum_r f_r}. \quad (5)$$

Counts and durations are pooled before division. The numerator uses the conventional  $n_r - 1$  output intervals; outputs arriving in one batch can have zero arrival gaps. The denominator excludes first-output latency and between-request agent/tool work. It is accumulated request time, which counts overlapping requests separately, not isolated GPU time or elapsed campaign time.

For Cqc10,  $N = 248,342$ ,  $R = 265$ ,  $\sum_r \ell_r = 10,667.6211$  seconds, and  $\sum_r f_r = 989.8239$  seconds yield 25.63 tokens/s. The 265 requests reconcile with 256 normal requests and nine successful internal compaction requests; all recorded completion reasons are stop. The ten pre/post metric brackets are contiguous. Generation-token and completed-request counters agree with their latency-histogram populations, with no counter reset or outstanding request at the selected boundaries.

The eligible SGLang EAGLE comparator, recorded on 24 August, serves the as-shipped Qwen3.8-27B NVFP4 checkpoint with FlashInfer, using speculative steps/top- $k$ /draft-token settings of 3/1/4. It uses the same Qwen Code version, agent harness, requested sampling settings, and network policy, with a 24,000-token response cap, a 262,144-token context limit, and an 8,192-token prefill chunk. Its two completed Astropy tasks are 12907 (resolved; 7.10 agent minutes) and 13033 (failed tests; 26.96 agent minutes). Their counters give  $N = 47,854$ ,  $R = 45$ ,  $\sum_r \ell_r = 2,013.6344$  seconds, and  $\sum_r f_r = 235.9488$  seconds, yielding 26.89 tokens/s. Both brackets contain streaming requests only. All completed engine requests are included, including internal agent traffic.

d) *Performance eligibility*: After inspecting the archived runs, we restrict the shared-task rate comparison to run-pairs in which both tasks completed and produced nonempty patches.

TABLE IV  
POOLED DECODE RATES FOR TREE VERIFICATION AND SGLANG EAGLE WITH QWEN3.8-27B NVFP4 ON GB10, USING EQ. 5. THE FIRST FOUR ROWS USE THE SAME TWO SWE-BENCH VERIFIED ASTROPY TASKS, 12907 AND 13033; EACH INCLUDED RUN PRODUCES NONEMPTY PATCHES ON BOTH TASKS; DECODER SETTINGS ARE SPECIFIED IN THE TEXT. CQC10 COVERS A SEPARATE TEN-TASK SUBSET. BOLD MARKS THE BEST RECORDED ELIGIBLE TREE RATE.

| Deployment          | Tasks | Requests | Tokens/s     |
|---------------------|-------|----------|--------------|
| SGLang EAGLE        | 2     | 45       | 26.89        |
| Tree Sr12 (Aug 19)  | 2     | 41       | <b>29.09</b> |
| Tree Cqc16 (Aug 19) | 2     | 35       | 28.28        |
| Tree Cqc15 (Aug 23) | 2     | 53       | 27.27        |
| Tree Cqc10 (Aug 24) | 10    | 265      | 25.63        |

We apply this retrospective rule to both systems and retain patches that fail tests. All three tree pairs and one of the two recovered SGLang pairs meet the rule. The other SGLang pair is excluded: task 13033 produced an empty patch after a final thinking-only response at the configured output cap, so the test harness was not invoked. Its failed outcome and original records remain in the artifact. We exclude the entire pair from this performance comparison, rather than retaining only its successful task. The rates therefore characterize patch-producing runs, not performance over all attempted tasks.

e) *Best recorded shared-task result*: We select task IDs 12907 and 13033 because both have completed SGLang records, then report all three recovered split-K tree runs on that pair. Sr12 and Cqc16 ran on 19 August; Cqc15 ran on 23 August. All three use the same tree geometry, full-vocabulary drafting, and grouped split-K attention configuration as Cqc10. Runtime checks confirm the specialized attention kernel in all sixteen full-attention layers, with no fallback. They share the loaded attention extension but differ in Python patch revisions. Each uses a 32,768-token response cap, compared with the eligible SGLang run’s 24,000. All 41 Sr12 responses and all 45 eligible SGLang responses are at most 20,000 tokens, below both configured caps; no response uses the differing output allowance. All Sr12 requests terminate with stop.

The best shared-task tree observation is Sr12:  $N = 25,027$ ,  $R = 41$ ,  $\sum_r \ell_r = 989.2612$  seconds, and  $\sum_r f_r = 130.4422$  seconds give **29.09 tokens/s**, 8.18% above the eligible SGLang rate under Eq. 5. Later observations are 28.28 for Cqc16 and 27.27 for Cqc15; Table IV retains them alongside the best result. All selected task brackets pass the same population and idle-boundary checks. Every row on the shared pair has one resolved task and one failed task. All six selected tree attempts used tools and produced nonempty patches, with no detected degeneration or malformed tool-call arguments under the recorded checks. Every selected tree request finished with stop, without a recorded response-cap hit.

The tree headline is its best recorded rate on the shared pair, not the latest run or Sr12’s entire workload. Sr12 completed four tasks in total, two resolved and two failed; pooling all four gives 26.21 tokens/s. Cqc16’s subsequent task 13236 exhibited degeneration, and Cqc15’s subsequent task 13398

lacks a completed evaluation. Those adverse records remain in the artifact and outside this explicitly two-task comparison. The shared-task comparison evaluates the complete decoding systems, including their proposal topology, draft budget, and serving optimizations. These method-specific choices are part of the comparison. The records use the same requested sampling settings and agent harness. The performance table retains every eligible run-pair: three tree observations and one SGLang observation. This retrospective subset does not establish performance over all attempts or each system’s optimum over draft lengths and candidate budgets. Repeated runs with a predeclared tuning protocol would measure that performance envelope; a component ablation would separately isolate the contribution of tree verification. The artifact retains the original metric brackets, source-bound configuration and behavior records, and reproducible pooled reductions.

*f) Selection and comparison limits:* Cqc10 contains the ten remaining tasks of an interrupted sixteen-task development campaign. Earlier segments changed source revisions and output caps and included degeneration, incomplete output, and budget-capped attempts. Source-bound audit records of those failures, retries, and missing evaluations remain in the artifact; the original records remain at their recorded source paths. We do not relabel the completed segment as the entire campaign, combine those segments into a single-configuration benchmark score, or drop unsuccessful attempts from a claimed benchmark denominator. The tasks were selected from one repository, and this deployment has one recorded boot without a matched ten-task native run. Separate targeted native probes change other serving settings and do not supply that comparator. Consequently, these records demonstrate an implemented verifier serving real coding-agent work, but establish no native-versus-tree task-speed or quality advantage. A current, matched task campaign is the remaining experiment for that claim.

*g) Existing comparator experiments:* Earlier SGLang EAGLE trials and native MTP controls remain in the repository. The earlier native campaign predates the NVFP4 deployment; later NVFP4 native probes cover targeted tasks with different serving settings. DSpark measurements use synthetic requests, and a later DFlash2 check establishes boot and response handling rather than agent-task performance. These records are distinguished from the pooled SWE comparison above; their superseded or non-agent rates are not additional performance results for the current verifier.

## B. Numerical and candidate-selection validation

The fused full-vocabulary selection test compares the two output tensors against the reference argmax and top-k operations. Across 1,368 input cases and five block-count settings (6,840 configurations), it records zero byte mismatches; planted negative controls fire. A separate captured four-level selection graph completes 24 replays with zero mismatches. These results validate candidate-selection outputs for the tested inputs and graph behavior, not end-to-end task equivalence [6].

Attention tests use the same kernel build as the ten-task deployment. They show bit-identical repeated output and log-

sum-exp values within each of sixteen determinism cases, with matching digests across two processes. On synthetic query/key/value tensors drawn at scales measured from banked model operands, 93.31% of output elements agree with the reference kernel within two ULP, the maximum absolute output difference is 0.00390625, and maximum log-sum-exp disagreement is four ULP. All nine declared checks pass, including comparisons against a float64 dense reference and checks for non-finite disagreements. These are attention-component bounds; split-K changes rounding and is not claimed to be byte-identical to the reference kernel or to preserve every downstream token.

Runtime checks confirm that the evaluated runs use the specialized attention kernel in all sixteen attention layers, fused selection, device acceptance, and captured state publication. The task outcomes and trace analysis in Section VII-A evaluate the composed system. These component and workload observations address different levels of behavior; neither establishes a full-model conditional-distribution proof.

## VIII. DISCUSSION AND REMAINING EVIDENCE

The design couples tree geometry, physical layout, and continuation publication. Optimizing any one of these in isolation can move work elsewhere or invalidate another stage’s mapping. The verifier uses transient cut states, static graph buffers, and native persistent rows for different purposes; conflating them would obscure both memory cost and state ownership.

Sequential replay and compact reconstruction remain alternative implementation choices. The present paper does not benchmark the authors’ Trees from Marginals, SpecLA, Bole, or TreeWY systems, and its component tests do not rank those methods. A direct comparison requires compatible models, matched workloads, and each system’s actual execution path.

For long-running agents, task completion and task wall time are the relevant application outcomes. More decoded tokens can reflect additional reasoning or a longer tool loop. The pooled decode comparison is therefore accompanied by task identities, outcomes, agent/tool policy, and terminal budgets. It describes the evaluated deployments and the stated retrospective patch-producing subset; it is not a task-completion speedup estimate.

Component validation and whole-task performance answer different questions. The selection test checks exact candidate outputs; the split-K test checks deterministic bounded rounding; the workload tests exercise the composed system. A broader quality claim requires held-out tasks and repetitions. Isolating an optimization’s application benefit requires toggling that component within the same eligible serving configuration and task protocol.

## IX. CONCLUSION

GDN Tree-Scan implements tree verification for a recurrent-hybrid model by connecting an ancestry-aware target forward to accepted-path state publication. A shared descriptor coordinates attention visibility, branch-local GDN computation, convolution

history, device probability arithmetic, and the next native continuation. Sequential scan and accepted-path replay make the state lifetime and arithmetic choices explicit; cached state, recomputation, physical layout, and independent-work batching expose the main optimization tradeoffs.

On two shared SWE-bench Verified Astropy tasks with Qwen3.8-27B NVFP4 on GB10, among runs producing nonempty patches on both tasks, the best recorded tree run attains 29.09 pooled tokens/s versus SGLang EAGLE’s 26.89, an 8.18% higher rate. Later tree runs on the same pair attain 28.28 and 27.27; the separate ten-task deployment attains 25.63. All ten attempts in that larger deployment terminated with nonempty patches and no degeneration detected by the recorded trace checks. The common rate definition compares the complete decoding systems; draft budgets, selection scope, and task outcomes are reported alongside the rates.

The contribution is a concrete verifier design and its implementation mechanisms, supported by component validation and workload-specific observations. A broader performance claim requires additional tasks and repeated runs with a declared tuning protocol. The present results do not establish broader task-quality preservation, full-model sequential equivalence, or general superiority over SGLang or the published compact-state systems.

#### APPENDIX A

##### REPRODUCIBILITY AND CLAIM BOUNDARIES

The artifact records source revisions, effective serving settings, kernel identities, tree geometry, graph and cache configuration, and original task metadata. Dated development records are retained separately from the evidence supporting the evaluated method.

All workload rates use the completed-request pooled decode estimator in Eq. 5. Token totals and latency sums come from the same completed-request population. First-output latency, tool execution, and evaluator time are separate quantities. Earlier global-token/subset-wall proxies and local-document timing do not supply paper performance claims. The selection ledger preserves every included and excluded run-pair and the unchanged unfiltered measurements.

*Algebraic correctness* concerns the real-arithmetic recurrence. *Numerical agreement* concerns particular kernels and references. *Sampler correctness* concerns the acceptance/residual law for the supplied probabilities. *Serving correctness* additionally requires coherent state across continuation and cache operations [2], [6], [25]. Equal candidate outputs, bounded attention error, and observed task completion address different parts of that chain.

For target probabilities  $p$  and proposal probabilities  $q$ , ordinary rejection sampling combines accepted mass  $\min(p, q)$  with residual mass proportional to  $(p - q)_+$ . This gives the intended target marginal only for the probabilities actually supplied [1], [2]. A correct abstract sampling rule cannot compensate for stale state or different target logits. The paper therefore does not promote component checks or task outcomes into a general distribution-preservation claim.

#### APPENDIX B

##### REMAINING WORKLOAD COMPARISONS

A matched native-MTP/tree comparison would freeze the checkpoint, task cohort, agent harness, sampling, tool policy, and budgets, then report outcomes and completion time over every attempted task. Each method may use its own speculative depth and serving optimizations; the tuning protocol and selected configuration must be explicit. Confirmation should predeclare performance eligibility and retain unsuccessful attempts in the outcome report.

Further component ablations, concurrent-agent experiments, and direct recurrent-verifier comparisons are conditional additions to that protocol. A mechanism comparison should include the chain-decomposition and replay overlaps in SpecLA and Trees from Marginals alongside Bole and TreeWY. Layout attribution needs a controlled layout intervention with unchanged logical candidates and correctly paired KV/mask remapping; its numerical diagnosis must remain separate from workload speed.

#### REFERENCES

- [1] Y. Leviathan, M. Kalman, and Y. Matias, “Fast Inference from Transformers via Speculative Decoding,” 2022. [Online]. Available: <https://arxiv.org/abs/2211.17192>
- [2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, “Accelerating Large Language Model Decoding with Speculative Sampling,” 2023. [Online]. Available: <https://arxiv.org/abs/2302.01318>
- [3] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wong, A. Zhu, L. Yang, X. Shi, C. Shi, Z. Chen, D. Arfeen, R. Abhyankar, and Z. Jia, “Specinfer: Accelerating generative large language model serving with tree-based speculative inference and verification,” 2024. [Online]. Available: <https://arxiv.org/abs/2305.09781>
- [4] S. Yang, J. Kautz, and A. Hatamizadeh, “Gated delta networks: Improving mamba2 with delta rule,” 2025. [Online]. Available: <https://arxiv.org/abs/2412.06464>
- [5] Y. Wu, Z. Qin, A. Wong, and S. Soatto, “Stree: Speculative tree decoding for hybrid state-space models,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.14969>
- [6] Z. Ma, “Lumo FlyWheel implementation and experiment archive,” [https://github.com/MaCoredroid/Lumo\\_FlyWheel/tree/984f613d7885d9bf7898866827552b68a3199d0c](https://github.com/MaCoredroid/Lumo_FlyWheel/tree/984f613d7885d9bf7898866827552b68a3199d0c), 2026, revision evidence snapshot; individual campaigns retain their own run identities.
- [7] —, “Stateless Tree Speculative Decoding: A Branched Verifier That Caches and Commits Like Native,” [https://macoredroid.github.io/Lumo\\_FlyWheel/stateless-tree.html](https://macoredroid.github.io/Lumo_FlyWheel/stateless-tree.html), 2026, author experiment archive; accessed 2026-09-21.
- [8] Z. Wang, X. Han, Z. Yang, F. Liu, X. Li, R. Gu, S. Zhong, and C. Tian, “SpecLA: Efficient Speculative Decoding for Linear-Attention Models,” 2026. [Online]. Available: <https://arxiv.org/abs/2607.16673v1>
- [9] Y. Oda, R. Mathieu, R. Knyazhitskiy, and A. Chakhvadze, “Trees from Marginals: Autoregressive drafting with factorized priors,” 2026, version 2, July 12, 2026; first submitted July 7, 2026. [Online]. Available: <https://arxiv.org/abs/2607.06763v2>
- [10] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” 2023. [Online]. Available: <https://arxiv.org/abs/2310.06770>
- [11] Z. Chen, A. May, R. Svirschevski, Y. Huang, M. Ryabinin, Z. Jia, and B. Chen, “Sequoia: Scalable, robust, and hardware-aware speculative decoding,” 2025. [Online]. Available: <https://arxiv.org/abs/2402.12374>
- [12] Y. Li, F. Wei, C. Zhang, and H. Zhang, “Eagle-2: Faster inference of language models with dynamic draft trees,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.16858>
- [13] J. Wang, Y. Su, J. Li, Q. Xia, Z. Ye, X. Duan, Z. Wang, and M. Zhang, “Opt-tree: Speculative decoding with adaptive draft tree structure,” 2025. [Online]. Available: <https://arxiv.org/abs/2406.17276>

- [14] Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test," 2025. [Online]. Available: <https://arxiv.org/abs/2503.01840>
- [15] J. Chen, Y. Liang, and Z. Liu, "DFlash: Block Diffusion for Flash Speculative Decoding," 2026. [Online]. Available: <https://arxiv.org/abs/2602.06036>
- [16] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," 2023. [Online]. Available: <https://arxiv.org/abs/2309.06180>
- [17] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient Execution of Structured Language Model Programs," 2023. [Online]. Available: <https://arxiv.org/abs/2312.07104>
- [18] R. Pan, Z. Wang, Z. Jia, C. Karakus, L. Zancato, T. Dao, Y. Wang, and R. Netravali, "Marconi: Prefix Caching for the Era of Hybrid LLMs," 2025. [Online]. Available: <https://arxiv.org/abs/2411.19379>
- [19] SGLang Team, "Hybrid Models Meet SGLang: More than Full Attention," <https://pytorch.org/blog/hybrid-models-meet-sglang-more-than-full-attention/>, 2025.
- [20] SGLang Project, "Qwen3-Next SGLang documentation," <https://lmsys.org/mintlify.app/cookbook/autoregressive/Qwen/Qwen3-Next>, 2026.
- [21] T. Dao, D. Haziza, F. Massa, and G. Sizov, "Flash-Decoding for long-context inference," <https://pytorch.org/blog/flash-decoding/>, 2023, october 13, 2023.
- [22] Z. Ye, L. Chen, R. Lai, W. Lin, Y. Zhang, S. Wang, T. Chen, B. Kasikci, V. Grover, A. Krishnamurthy, and L. Ceze, "FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving," 2025. [Online]. Available: <https://arxiv.org/abs/2501.01005>
- [23] J. Yao, K. Chen, K. Zhang, J. You, B. Yuan, Z. Wang, and T. Lin, "DeFT: Decoding with Flash Tree-attention for Efficient Tree-structured LLM Inference," 2025. [Online]. Available: <https://arxiv.org/abs/2404.00242>
- [24] T. Dao, "Flashattention-2: Faster attention with better parallelism and work partitioning," 2023. [Online]. Available: <https://arxiv.org/abs/2307.08691>
- [25] S. Yang, B. Wang, Y. Zhang, Y. Shen, and Y. Kim, "Parallelizing Linear Transformers with the Delta Rule over Sequence Length," 2024. [Online]. Available: <https://arxiv.org/abs/2406.06484>
- [26] Y. Wu, Y. Dukler, M. Trager, A. Achille, W. Xia, and S. Soatto, "Snakes and Ladders: Accelerating SSM Inference with Speculative Decoding," in *Proceedings of the 4th NeurIPS Efficient Natural Language and Speech Processing Workshop*, vol. 262. PMLR, 2024, pp. 292–304. [Online]. Available: <https://proceedings.mlr.press/v262/wu24a.html>
- [27] L. Zhong, S. Xu, H. Wen, T. Xie, Q. Guo, Y. Wang, and M. Li, "Specmamba: Accelerating mamba inference on fpga with speculative decoding," 2025. [Online]. Available: <https://arxiv.org/abs/2509.19873>
- [28] H. Borobia, E. Seguí-Mas, and G. Tormo-Carbó, "Component-aware self-speculative decoding in hybrid language models," 2026. [Online]. Available: <https://arxiv.org/abs/2605.01106>
- [29] L. Wang, Y. Su, X. Wu, C. You, Y. Liu, Z. Qiu, J. Zhang, J. Zheng, F. Liu, J. Zhang, C. Tian, and C. Huan, "Bole: Efficient Tree Speculation for Hybrid-Attention Language Models," 2026. [Online]. Available: <https://arxiv.org/abs/2608.01651>
- [30] S. M. Ghantasala, "TreeWY: Speculative Verification for Gated DeltaNet Hybrids," 2026. [Online]. Available: <https://arxiv.org/abs/2608.20961>
- [31] Dao AI Lab and NVIDIA, "ReplaySSM: Cache SSM Inputs Instead of State," <https://github.com/vllm-project/vllm/issues/47572>, 2026, implementation RFC; accessed 2026-09-21.
- [32] H. He, "Defeating Nondeterminism in LLM Inference," <https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/>, 2025, thinking Machines Lab, September 10, 2025.
- [33] G. Oliaro, Z. Jia, D. Campos, and A. Qiao, "SuffixDecoding: Extreme Speculative Decoding for Emerging AI Applications," 2025. [Online]. Available: <https://arxiv.org/abs/2411.04975v3>
- [34] Snowflake Arctic Inference, "Suffix Decoding," <https://arcticinference.readthedocs.io/en/latest/suffix-decoding.html>, 2026, implementation documentation; accessed September 24, 2026.
- [35] Y. Hu, K. Wang, X. Zhang, F. Zhang, C. Li, H. Chen, and J. Zhang, "SAM Decoding: Speculative Decoding via Suffix Automaton," 2024. [Online]. Available: <https://arxiv.org/abs/2411.10666v3>